SmartSolar — Complete Project Summary

Chunk-by-Chunk Walkthrough of `solar_pipeline.py` and `app.py`

> **ELI5** = Explain Like I'm 5 — used at complex points to make concepts crystal-clear.

Table of Contents

1. Project Overview
2. How the Two Files Work Together
3. solar_pipeline.py — Chunk by Chunk
    - Chunk 1 — Imports
    - Chunk 2 — Global Constants
    - Chunk 3 — Data Generation (The Most Critical Chunk)
    - Chunk 4 — Exploratory Data Analysis (EDA)
    - Chunk 5 — Pipeline Diagram
    - Chunk 6 — Preprocessing
    - Chunk 7 — Metric Helper (MAPE Fix)
    - Chunk 8 — Train 5 (or 7) ML Models
    - Chunk 9 — Evaluation Charts
    - Chunk 10 — Final Summary
4. app.py — Chunk by Chunk
    - Chunk 1 — Imports
    - Chunk 2 — Page Config & CSS
    - Chunk 3 — Constants (Must Match Pipeline)
    - Chunk 4 — Load Models & Data (Cached)
    - Chunk 5 — Core Prediction & Physics Helpers
    - Chunk 6 — Sidebar (All User Inputs)
    - Chunk 7 — Main Panel Header & Guard
    - Chunk 8 — Build Input Dicts & Run Predictions
    - Chunk 9 — Key Metrics Row
    - Chunk 10 — Real-World Loss Factor Breakdown
    - Chunk 11 — Gauges, Temperature, Backup Breakdown
    - Chunk 12 — Analysis Tabs
5. Parameter Cascade — How Changing One Input Affects Everything
6. The Three Core Fixes Explained
7. Data Flow Diagram (End-to-End)
8. File & Folder Structure

---

1. Project Overview

SmartSolar is a capstone project that predicts real-world solar panel DC power output using machine learning, then displays the results in a Streamlit web dashboard.

The core problem it solves:
The standard physics formula for solar panels — `P = η × A × G × [1 − β(T_mod − 25)]` — predicts *ideal* output under perfect lab conditions. Real panels are dirty, aging, shaded by trees, and have wiring losses. These factors can cut output by **15–50%** and the formula has no way to model them. ML learns these real-world deviations from training data and corrects for them.

ELI5: Imagine you have a car that the manufacturer says gets 50 km/L. But in real life, with traffic, hills, wind, and a dirty air filter, you only get 35 km/L. The formula is the manufacturer's claim. ML is the mechanic who has watched thousands of cars in real conditions and can tell you what you'll *actually* get.

Two-file architecture:

File	Purpose	When to run
`solar_pipeline.py`	Generates data, trains 5–7 ML models, saves all `.pkl` files	Once (or when retraining)
`app.py`	Loads saved models, serves the Streamlit dashboard	Every session via `streamlit run app.py`

---

2. How the Two Files Work Together

```
solar_pipeline.py  ──runs once──►  data/solar_dataset.csv
                                   models/scaler.pkl
                                   models/rf_model.pkl
                                   models/gb_model.pkl
                                   models/mlp_model.pkl
                                   models/xgb_model.pkl  (or svr_model.pkl)
                                   models/lgb_model.pkl  (or et_model.pkl)
                                   outputs/*.png  (charts)

app.py  ──loads from disk──►  all of the above
        ──serves──►  Streamlit web UI at http://localhost:8501
```

Critical contract: Both files must share:

- The same `FEATURES` list (12 features, same order)
- The same `MAPE_THRESHOLD = 50.0`
- The same `scaler.pkl` (so app inputs are scaled identically to training inputs)

If you change `FEATURES` in the pipeline, you *must* retrain (`python solar_pipeline.py`) before running the app again.

---

3. `solar_pipeline.py` — Chunk by Chunk

Chunk 1 — Imports

```
import os, time, warnings
import numpy as np
import pandas as pd
import matplotlib, matplotlib.pyplot as plt, matplotlib.patches,
matplotlib.gridspec
import joblib
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor,
ExtraTreesRegressor
from sklearn.neural_network import MLPRegressor
from sklearn.svm import SVR
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
# Optional (falls back if not installed):
from xgboost import XGBRegressor
from lightgbm import LGBMRegressor
```

What it does: Loads every library needed for the whole pipeline. `matplotlib.use("Agg")` is important — it tells matplotlib to render charts to files (PNG) rather than a GUI window, because this script runs headless (no screen needed).

XGBoost and LightGBM are wrapped in `try/except` so the pipeline still runs if they aren't installed — it falls back to SVR and Extra Trees respectively.

Chunk 2 — Global Constants

```
SEED        = 42        # Makes randomness reproducible
N_SAMPLES   = 50_000    # 50,000 synthetic training rows
TRAIN_FRAC  = 0.80      # 80% train, 20% test
MAPE_THRESHOLD = 50.0   # Only compute MAPE on outputs > 50W (FIX 3)

FEATURES = [
    "irradiance", "ambient_temp", "module_temp",
    "humidity", "wind_speed", "hour", "month",
    "day_of_year", "efficiency", "panel_area",
    "temp_coeff", "noct",
]
TARGET = "dc_power"
```

What it does: Sets the "rules" for the entire pipeline. Every important parameter lives here so it's easy to change in one place.

ELI5 — why SEED=42?
Random number generators need a starting point ("seed"). If you always give them the same starting point (42), you get the same random numbers every run. This means your results are reproducible — someone else running your code gets the exact same dataset and model scores. Without a seed, every run gives different results, making it impossible to compare experiments.

Why 50,000 rows?
ML models need enough data to learn patterns. 50,000 rows gives enough coverage of all possible combinations of irradiance, temperature, humidity, shading events, and panel types without being so large that training takes hours.

Chunk 3 — Data Generation (The Most Critical Chunk)

This is the heart of the project. It creates all 50,000 training rows from scratch using physics equations plus controlled randomness.

Step-by-step inside `generate_dataset()`:

① Time features

```
hour  = np.random.randint(0, 24, n)   # random hour 0–23
doy   = np.random.randint(1, 366, n)  # random day of year 1–365
month = ((doy - 1) // 30 + 1).clip(1, 12)  # derived from doy
```

Randomly picks hours and days to represent a whole year of operation.

② Irradiance (G in W/m²) — how much sunlight hits the panel

```
daylight = (hour >= 6) & (hour <= 18)     # only generate sunlight 6am–6pm
G_raw    = 1000.0 * sin(π × (hour-6)/12) # peaks at noon (hour=12), zero at
6am/6pm
seasonal = 0.75 + 0.25 × sin(2π(doy-80)/365) # summer brighter than winter
cloud    = uniform(0.30, 1.20)             # cloud cover: dims OR concentrates
light
G        = clip(G_raw × seasonal × cloud, 0, 1100)
```

ELI5: Think of irradiance like the brightness of sunlight. It follows a natural arc — low at sunrise, high at noon, low at sunset. This code recreates that arc with math (the `sin` function makes the smooth curve). Then it adds seasons (summer sun is stronger) and clouds (random multiplier that can dim or briefly concentrate light).

③ Ambient Temperature

```
T_amb = 22.0 + 8.0×sin(seasonal) + 5.0×sin(diurnal) + Normal(0, 1.5)
```

Temperature follows: a year-round baseline (22°C) + seasonal variation (±8°C, warmer in summer) + daily variation (±5°C, warmer at noon) + random noise (±1.5°C sensor jitter).

④ Module Temperature — the NOCT Model (IEC 61215 standard)

```
T_mod = T_amb + (noct - 20.0) / 800.0 * G
```

ELI5 — What is NOCT?
NOCT = Nominal Operating Cell Temperature. Panels heat up in the sun beyond the air temperature around them. The NOCT value (typically 43–48°C) tells you how much hotter the panel gets in standard test conditions. This formula says: *"the panel is always hotter than the air, and how much hotter depends on how much sunlight is falling on it."*

Example: Air = 30°C, G = 800 W/m², NOCT = 45°C
→ T_mod = 30 + (45-20)/800 × 800 = 30 + 25 = **55°C**
The panel is 25°C hotter than the air!

⑤ The Ideal Physics Formula (the baseline)

```
P_ideal = eta × area × G × (1.0 - beta × (T_mod - 25.0))
```

Breaking each part down:

Term	Meaning
`eta` (η)	Panel efficiency — what fraction of sunlight becomes electricity (e.g. 0.20 = 20%)
`area`	Panel surface area in m²
`G`	Irradiance — sunlight power hitting the panel in W/m²
`1 - beta × (T_mod - 25)`	Temperature derating — panels lose efficiency as they get hotter

ELI5 — The temperature derating part:
Panels work best at 25°C (lab conditions). Every degree above 25°C costs you `beta`% of efficiency. Beta is typically 0.3–0.5%/°C. So if your panel is at 55°C (30°C above STC), and beta = 0.4%/°C, you lose 30 × 0.4% = 12% of your power. This is why desert installations with high-efficiency panels still underperform in midsummer heat.

⑥ FIX 1 — Real-World Loss Factors (why ML is necessary)

This is what makes the project academically valid. Without this step, the ML models would just be learning to approximate a formula — which is pointless because you could just use the formula directly.

```
soiling_factor = uniform(0.80, 1.00)      # dust/pollution: 0–20% loss
aging_factor   = uniform(0.92, 1.00)      # panel age: 0–8% loss
shade_event    = random < 0.12             # 12% chance of shading
shading_factor = where(shade_event, uniform(0.50, 0.85), 1.0)  # 15–50% loss
wiring_factor  = uniform(0.95, 0.99)      # cable/mismatch: 1–5% loss
```

```
P_dc = P_ideal × soiling × aging × shading × wiring + Normal(0, 5)
```

ELI5 — Why do these matter?

Loss Factor	What it means in plain English
Soiling (0.80–1.00)	Dust, bird droppings, and pollution coat the panel glass. Blocks 0–20% of sunlight. In a desert city, 15–20% soiling loss is common. After rain, it resets to near 1.0.
Aging (0.92–1.00)	Every panel slowly degrades. After 16 years at 0.5%/year, a panel is 92% of its original rated output. Older panels = lower factor.
Shading (0.50–0.85)	When even ONE cell in a panel is in shade (from a tree, chimney, neighboring building), the bypass diodes activate and the *entire string* drops. This is highly non-linear — the formula cannot model it at all. Happens 12% of the time in this dataset.
Wiring (0.95–0.99)	DC cables have electrical resistance. When current flows through resistance, power is lost as heat (I^2R loss). Panels in a string are also never perfectly matched — the weakest panel limits the whole string.

Combined effect example:

- `P_ideal` = 1000 W
- soiling = 0.90 (10% dusty)
- aging = 0.96 (4-year-old panel)
- shading = 0.70 (partially shaded)
- wiring = 0.97
- `P_dc` = 1000 × 0.90 × 0.96 × 0.70 × 0.97 ≈ **589 W** — only 59% of ideal!

The physics formula cannot produce this 589W — it would say 1000W. ML learns this gap from 50,000 examples and can correctly predict ~589W.

The DataFrame has these columns:

Column	Used in training?	Purpose
`irradiance`, `ambient_temp`, … (12 cols)	✅ Yes — features	Inputs to the model
`dc_power`	✅ Yes — target	What we predict
`p_ideal`	❌ No — diagnostic	Shows what formula alone would predict
`combined_loss_factor`	❌ No — diagnostic	Tracks total loss for EDA
`is_shaded`	❌ No — diagnostic	Flags shaded rows

Chunk 4 — Exploratory Data Analysis (EDA)

What it does: Creates 3 matplotlib figure files saved to `outputs/`:

Figure	File	What it shows
Figure A	`eda_A_distributions.png`	9 subplots: distribution of DC power, loss factors, hourly profile, monthly averages, shaded vs unshaded, temperature heating effect
Figure B	`eda_B_correlations.png`	6 subplots: correlation heatmap, irradiance vs power scatter, temperature derating curves, efficiency groups, formula-vs-real gap, humidity effect
Figure C	`eda_C_statistics.png`	Statistical summary table (mean, std, min, max, skewness) for all features

Key insight from Figure A, subplot A1: A scatter plot of `p_ideal` vs `dc_power`. A perfect formula would show all dots on the diagonal line. Instead, dots scatter *below* the diagonal — real output is always less than ideal. This is the visual proof that ML is needed.

Key insight from Figure B, subplot B5: Shows the "gap %" between formula and actual, broken down by irradiance level. The gap is approximately constant (not irradiance-dependent) because the loss factors are independent of sunlight level. This validates that the loss factors are realistic (soiling doesn't depend on how sunny it is).

Chunk 5 — Pipeline Diagram

What it does: Creates `outputs/pipeline_diagram.png` — a visual flowchart showing the entire ML pipeline architecture, from inputs through dataset creation, EDA/preprocessing, model training, evaluation, and finally to the Streamlit app.

This is primarily for presentation/documentation purposes. The code uses matplotlib to draw colored boxes and arrows.

Chunk 6 — Preprocessing

```
split_idx  = int(len(df) * 0.80)     # row 40,000 = train/test boundary
train_df   = df.iloc[:split_idx]      # rows 0–39,999
test_df    = df.iloc[split_idx:]      # rows 40,000–49,999

# Remove night rows from TRAINING only (irradiance = 0 → trivially P_dc = 0)
train_df = train_df[train_df["irradiance"] > 0]

X_train = train_df[FEATURES].values
y_train = train_df[TARGET].values

# Scale features 0–1 using MinMaxScaler
scaler     = MinMaxScaler()
```

```
X_train_sc = scaler.fit_transform(X_train)  # fit AND transform
X_test_sc  = scaler.transform(X_test)       # transform ONLY (no fit!)

joblib.dump(scaler, "models/scaler.pkl")    # save for app.py to use
```

Why remove night rows from training but NOT from test?
During training, nighttime rows (irradiance = 0) always produce P_dc ≈ 0W. There's nothing to learn from them — they would just waste training time and add noise. But in the test set, we keep them to get a realistic picture of model performance across all 24 hours.

ELI5 — What is MinMaxScaler?
Different features have wildly different ranges: irradiance goes 0–1100, but `hour` only goes 0–23, and `efficiency` is 0.14–0.22. If you feed these raw numbers into a neural network or SVM, the large numbers (irradiance) will dominate and the model ignores small numbers. MinMaxScaler squashes every feature to the range 0–1 so the model treats them equally.

Before scaling	After scaling
irradiance: 750 W/m²	0.682
hour: 14	0.609
efficiency: 0.20	0.750

Critical rule — Never fit the scaler on test data!
If you fit the scaler on the test set, the test data "leaks" into the scaler (it adjusts min/max to include test values). This inflates performance scores. You must only `fit` on training data, then `transform` both sets with the same fitted scaler.

Chunk 7 — Metric Helper (MAPE Fix)

```
def compute_metrics(y_true, y_pred):
    r2   = r2_score(y_true, y_pred)
    rmse = sqrt(mean_squared_error(y_true, y_pred))
    mae  = mean_absolute_error(y_true, y_pred)

    # FIX 3: compute MAPE only on rows where actual > 50W
    mask = y_true > 50.0
    mape = mean(|y_true[mask] - y_pred[mask]| / y_true[mask]) × 100

    return {R2, RMSE, MAE, MAPE}
```

Why is MAPE special here?
MAPE = Mean Absolute Percentage Error. For nighttime/dawn/dusk rows where actual output is tiny (e.g. 3W), even a "good" prediction of 5W gives: `|3-5|/3 = 67% error`. This is mathematically correct but meaningless for energy planning — nobody cares about 3W predictions.

Before FIX 3: MAPE was computed on all rows where actual > 1W, leading to **MAPE values of 130–155%**. This looked terrible and was misleading. After FIX 3: MAPE is only computed on rows where actual > **50W** (productive daylight hours). This gives **realistic MAPE values of 8–15%**.

Metric	What it measures	Good value
R^2	What % of output variance is explained by the model (0–1 scale)	> 0.95
RMSE	Root Mean Squared Error — average error in Watts, penalizes large errors more	< 30W
MAE	Mean Absolute Error — average absolute error in Watts, treats all errors equally	< 20W
MAPE (>50W)	Average % error on productive daytime outputs only	< 15%

Chunk 8 — Train 5 (or 7) ML Models

Each model is configured with specific hyperparameters, then trained on `X_train_sc` / `y_train`:

Model	Key Hyperparameters	Character
Random Forest	300 trees, max depth 16, 70% features per tree	Ensemble of decision trees, votes average. Robust to noise, gives feature importance.
Gradient Boosting	400 trees, learning rate 0.05, max depth 6	Builds trees sequentially, each one fixes errors of the last. Slower but very accurate.
MLP / ANN	3 hidden layers (256→128→64), ReLU, Adam optimizer, 600 max iterations	Neural network. Learns non-linear patterns. Early stopping prevents overfitting.
XGBoost	400 trees, L1+L2 regularization	Optimized gradient boosting. Usually the fastest top-performer. Falls back to SVR if not installed.
LightGBM	400 trees, 63 leaves, leaf-wise growth	Even faster gradient boosting. Falls back to Extra Trees if not installed.

ELI5 — How does an ensemble of trees work?
A single decision tree asks a series of yes/no questions ("Is irradiance > 500? Is module_temp > 45?...") until it reaches a prediction. One tree is brittle and overfits. Random Forest grows 300 different trees, each trained on a random subset of data, then **averages all their predictions**. Averaging reduces errors dramatically — the trees' mistakes cancel each other out.

Training loop:

```
for name, model in model_registry.items():
    model.fit(X_train_sc, y_train)        # train
    preds = model.predict(X_test_sc).clip(0)  # predict on test, floor at 0W
```

```
    metrics = compute_metrics(y_test, preds)  # score
    joblib.dump(model, f"models/{filename}")   # save to disk
```

The best model (highest R^2) is also saved as `models/best_model.pkl`.

Chunk 9 — Evaluation Charts

Creates 4 matplotlib figures saved to `outputs/`:

Figure	File	Contents
Figure D	`model_comparison.png`	8-panel dashboard: R^2 bar chart, training time, RMSE, MAPE, radar chart, actual vs predicted scatter, residual histogram, feature importance
Figure E	`results_table.png`	Ranked results table for all models with R^2, RMSE, MAE, MAPE, training time
Figure F	`residual_analysis.png`	Residual histogram for every model side by side
Figure G	`formula_vs_ml.png`	FIX 2 validation: physics formula predictions vs ML predictions vs actual — proves ML is necessary

Key interpretation guidance:

- **Actual vs Predicted (D6):** Dots should cluster tightly around the diagonal. Scatter = error.
- **Residuals (D7/F):** Should form a bell curve centered at 0W. A mean near 0W means the model is unbiased (not systematically over- or under-predicting).
- **Feature Importance (D8):** Shows which of the 12 input features the Random Forest found most useful. Irradiance is almost always #1 (it has the biggest single effect on output). Efficiency and panel area are also high.

Chunk 10 — Final Summary

Prints a formatted summary box to the console showing all model rankings, the best model's metrics, and a reminder to run the Streamlit app. This is purely informational output — it doesn't write any files.

---

4. `app.py` — Chunk by Chunk

Chunk 1 — Imports

```
import os, math
import joblib, numpy, pandas, streamlit
import plotly.graph_objects, plotly.express
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
```

Uses **Plotly** instead of matplotlib for interactive charts (hover tooltips, zooming). `math` is used for the hourly irradiance simulation (sine function for sun angle).

Chunk 2 — Page Config & CSS

```
st.set_page_config(
    page_title="SmartSolar",
    page_icon="🌞",
    layout="wide",              # full browser width
    initial_sidebar_state="expanded",
)
st.markdown("""<style>...""")  # custom CSS
```

This must be the very first Streamlit call — Streamlit raises an error if any other `st.*` call comes before `set_page_config`.

The custom CSS defines:

- `.calc-box` — blue-bordered monospace boxes for formula display
- `.loss-box` — amber-bordered boxes for loss factor display
- `div[data-testid="metric-container"]` — styles the metric cards with a light blue tint and rounded border

---

Chunk 3 — Constants (Must Match Pipeline)

```
MAPE_THRESHOLD = 50.0   # MUST match solar_pipeline.py

FEATURES = [
    "irradiance", "ambient_temp", "module_temp",
    "humidity", "wind_speed", "hour", "month",
    "day_of_year", "efficiency", "panel_area",
    "temp_coeff", "noct",
]

MODEL_FILES = {
    "MLP / ANN"        : "models/mlp_model.pkl",
    "Gradient Boosting": "models/gb_model.pkl",
    "Random Forest"    : "models/rf_model.pkl",
    "XGBoost"          : "models/xgb_model.pkl",
    "LightGBM"         : "models/lgb_model.pkl",
    "SVR (RBF)"        : "models/svr_model.pkl",
    "Extra Trees"      : "models/et_model.pkl",
}
```

Why must these match the pipeline exactly?
The `FEATURES` list defines the **order** in which values are assembled into the input vector before being passed

to the scaler and then the model. If the app uses a different order than what the scaler was fit on, every scaled value is mapped to the wrong feature → completely wrong predictions.

The `MODEL_FILES` dict tells the app which `.pkl` files to load. Only files that actually exist on disk get loaded (the `if os.path.exists(path)` check in the next chunk).

Chunk 4 — Load Models & Data (Cached)

```
@st.cache_resource      # loads once, persists across reruns
def load_assets():
    scaler = joblib.load("models/scaler.pkl")
    models = {name: joblib.load(path) for name, path in MODEL_FILES.items()
              if os.path.exists(path)}
    return scaler, models

@st.cache_data          # loads once, caches the DataFrame
def load_training_data():
    return pd.read_csv("data/solar_dataset.csv")
```

ELI5 — What does `@st.cache_resource` do?
Every time you move a slider in the app, Streamlit re-runs the *entire* `app.py` script from top to bottom. Without caching, it would reload all 7 model `.pkl` files (each potentially 100MB+) on every slider move — making the app freeze for several seconds. `@st.cache_resource` tells Streamlit: "after the first load, remember the result and reuse it. Never reload unless the code changes."

`@st.cache_data` does the same for DataFrames (uses memory-efficient serialization).

If the `.pkl` files don't exist (pipeline hasn't been run), a `FileNotFoundError` is caught and `ALL_OK = False` is set. The app then shows an error message in Chunk 7.

Chunk 5 — Core Prediction & Physics Helpers

Four functions do all the physics and ML math:

`noct_module_temp(T_amb, G, noct)`

```
T_mod = T_amb + (noct - 20) / 800 × G
```

Same NOCT formula as the pipeline. Keeps physics consistent between training and inference.

`physics_formula(eta, area, G, beta, T_mod)`

```
P_ideal = max(0, η × A × G × [1 − β × (T_mod − 25)])
```

The baseline formula. Used for comparison only — displayed alongside ML predictions to show the gap ML closes.

`predict_all_models(panel, weather)`

```
row   = [G, T_amb, T_mod, humidity, wind_speed, hour, month, doy,
         efficiency, panel_area, temp_coeff, noct]
x_sc  = scaler.transform([row])          # scale using saved scaler
preds = {name: max(0, model.predict(x_sc)[0]) for name, model in models.items()}
```

Assembles the 12 features in the exact same order as training, scales them with the saved scaler, runs every loaded model, floors all outputs at 0W (no negative power), and returns them all.

ELI5 — why `max(0, ...)`?
ML models can sometimes predict negative numbers (e.g. -5W at night). Negative power output is physically impossible. Clipping at 0 is called "domain-aware post-processing."

`compute_backup(P_dc, panel)`

```
P_ac    = P_dc × inverter_efficiency
E_panel = P_dc × peak_sun_hours
E_batt  = battery_Wh × depth_of_discharge × inverter_efficiency
E_total = E_panel + E_batt
backup  = E_total / load_W   (hours)
```

ELI5 — Backup runtime calculation:

- `E_panel` = how many Watt-hours your panels generate today
- `E_batt` = how many usable Watt-hours are stored in the battery (limited by DoD — you can't drain most batteries to 0% without damaging them)
- `E_total` = total energy available
- `backup` = total energy ÷ power demand = hours you can run at that load

Example: E_total = 3000 Wh, load = 300W → 10 hours of backup

Chunk 6 — Sidebar (All User Inputs)

The sidebar is divided into 5 sections:

Section 1: Panel Preset

```
preset = st.selectbox("Load a real panel spec", ["Custom", "Jinko Solar 400Wp...",
...])
PRESETS = {
    "Jinko Solar 400Wp Mono (2024)": dict(rated=400, eff=20.5, area=1.947,
```

```
    beta=0.35, noct=45.0),
        ...11 more presets...
    }
    p = PRESETS.get(preset, {})  # {} if "Custom" selected
```

Selecting a preset auto-fills the panel spec inputs below with real manufacturer datasheet values. "Custom" leaves the defaults unchanged.

Section 2: Panel Specifications

Input	Type	Range	What it controls
Rated Power (Wp)	number_input	50–800	Status label calculation (pct_of_rated)
Panel Efficiency (%)	number_input	10–24	Core physics formula η
Panel Area (m^2)	number_input	0.5–3.0	Core physics formula A
Temp Coefficient (%/°C)	number_input	0.20–0.60	Thermal loss per °C above 25°C
NOCT (°C)	number_input	40–50	Module temperature calculation
Number of Panels	number_input	1–200	Multiplies panel_area in predictions

Section 3: Real-World Condition Estimates (FIX 1)

Slider	Range	Default	Effect
Soiling loss (%)	0–25	5	Added to total_loss_pct
Panel aging loss (%)	0–12	3	Added to total_loss_pct
Wiring + mismatch loss (%)	1–6	2	Added to total_loss_pct
Partial shading loss (%)	0–50	0	Added to total_loss_pct

```
total_loss_pct = soiling + aging + wiring + shading  # e.g. 5+3+2+0 = 10%
combined_factor = max(0, (100 - total_loss_pct) / 100)  # e.g. 0.90
```

`combined_factor` is applied as a multiplier on the ML ensemble output in Chunk 8.

Section 4: System Specifications

Slider/Input	What it controls
Inverter Efficiency (%)	AC output = DC × this
Battery Capacity (Wh)	Energy stored
Battery DoD (%)	Usable fraction of battery

Slider/Input	What it controls
Load Demand (W)	Divides total energy for backup hours
Peak Sun Hours / Day	Multiplies DC power for daily energy

Section 5: Current Weather Conditions

Slider	Usage
Solar Irradiance (W/m^2)	Direct model input + irradiance sensitivity tab
Ambient Temperature (°C)	NOCT calculation → T_mod
Humidity (%)	Direct model input
Wind Speed (m/s)	Direct model input
Hour of Day	Direct model input + hourly profile tab
Month	Direct model input + seasonality
Day of Year	Direct model input + hourly profile tab

Chunk 7 — Main Panel Header & Guard

```
st.title("🌞 SmartSolar: Predicting Solar Panel Output...")

if not ALL_OK:
    st.error("⚠️ Run `python solar_pipeline.py` first.")
    st.stop()  # halts all further rendering
```

`st.stop()` is a Streamlit special function that prevents the rest of the page from rendering. Without this, the app would crash with cryptic errors about missing variables when model files don't exist.

Chunk 8 — Build Input Dicts & Run Predictions

```
panel_spec = {
    "efficiency"  : eff_pct / 100.0,       # convert 20.5% → 0.205
    "panel_area"  : panel_area * num_panels, # scale by number of panels
    "temp_coeff"  : beta_pct / 100.0,       # convert 0.35% → 0.0035
    "noct"        : noct_val,
    ...
}
weather_now = {
    "irradiance"  : float(irradiance),
    "ambient_temp": float(ambient_temp),
    ...
}
```

```
all_preds, T_mod = predict_all_models(panel_spec, weather_now)
ens_pdc          = np.mean(list(all_preds.values()))  # ensemble average
adjusted_pdc     = ens_pdc * combined_factor           # apply site losses
bk               = compute_backup(adjusted_pdc, panel_spec)
p_ideal          = physics_formula(...)                # baseline comparison
```

Key understanding — two layers of loss:

1. The **ML model** already learns statistical real-world loss patterns from 50,000 training samples. Its prediction includes an "average" level of soiling/aging/shading.
2. The **user-configured loss sliders** let users tune for *their specific site*. If your site has unusually heavy pollution (20% soiling), you add that on top to account for your specific conditions.

This two-layer approach is physically honest: ML learns the underlying patterns; users fine-tune for their installation.

Status label:

```
pct_of_rated = adjusted_pdc / (rated_wp × num_panels) × 100
if pct_of_rated >= 70:  → "🟢 Good output"
elif pct_of_rated >= 40: → "🟡 Moderate"
else:                    → "🔴 Low output"
```

Chunk 9 — Key Metrics Row

```
cols = st.columns(len(all_preds) + 3)
for i, (name, pdc) in enumerate(all_preds.items()):
    cols[i].metric(name, f"{pdc:.0f} W")
cols[-3].metric("🔗 ML Ensemble",      f"{ens_pdc:.0f} W")
cols[-2].metric("⚡ Adjusted Output",  f"{adjusted_pdc:.0f} W", delta="-X% real-
world losses")
cols[-1].metric("⏱️ Backup Runtime",   f"{bk['backup']:.1f} hrs")
```

Displays a row of metric cards across the full page width. Shows:

- Each individual model's raw prediction
- The ensemble average (average of all models)
- The adjusted prediction (ensemble × combined_factor)
- Backup runtime in hours

Why use an ensemble (average of all models)?
Each model has different strengths. Random Forest is robust to outliers. Gradient Boosting corrects systematic errors. MLP captures non-linear interactions. Averaging their predictions reduces the risk that any single model's weakness dominates — this is called **wisdom of the crowd** in ML.

Chunk 10 — Real-World Loss Factor Breakdown

```
with st.expander("📉 Real-World Loss Breakdown — X% total site derating"):
    lc1.metric("Physics formula", f"{p_ideal:.0f} W", help="Ideal — no losses")
    lc2.metric("Soiling loss",    f"-{soiling_pct}%", delta=f"-
{p_ideal*soiling/100:.0f} W")
    lc3.metric("Aging loss",      f"-{aging_pct}%",   delta=...)
    lc4.metric("Wiring",          f"-{wiring_pct}%",  delta=...)
    lc5.metric("Shading",         f"-{shading_pct}%", delta=...)
```

Shows exactly how many Watts are lost at each stage, starting from the physics formula (ideal) and applying each loss factor in sequence. This makes the gap between "what the formula says" and "what you'll actually get" transparent and interpretable.

Chunk 11 — Gauges, Temperature, Backup Breakdown

Three side-by-side visualizations:

Output Gauge (Plotly Indicator):
A speedometer-style gauge with three colored zones:

- Red zone (0–40% of rated)
- Yellow zone (40–70% of rated)
- Green zone (70–105% of rated)

The needle shows `adjusted_pdc`. The delta shows the gap between the physics formula and adjusted output (i.e., how much ML+losses corrected the ideal prediction).

Module Temperature display:
Shows the calculated T_mod with color coding:

- Green = below 48°C (cool, good)
- Amber = 48–65°C (warm, some loss)
- Red = above 65°C (hot, significant loss)

Thermal loss % is displayed: `temp_loss_pct = beta_pct × max(0, T_mod - 25) × 100`

ELI5 — Why does temperature matter so much?
Imagine a solar panel like a person jogging. In cool weather (25°C), they run efficiently. In extreme heat (55°C), they slow down — just from the heat. Panels have the same problem: every degree above 25°C (the lab test temperature) causes them to lose some efficiency. beta = 0.4%/°C means at T_mod = 65°C (40°C above STC), the panel loses 16% of its rated output just from heat.

Backup Runtime Bar Chart:
Shows three bars: Panel energy today (Wh), Usable battery (Wh), Total available (Wh).

Chunk 12 — Analysis Tabs

Five interactive analysis tabs:

Tab 1: ⏰ Today's Hourly Profile

Simulates how output changes across all 24 hours of the current day:

```
for h in range(24):
    G_h = irradiance × sin(π(h-6)/12) × seasonal_factor  # sun angle arc
    w_h = {**weather_now, "irradiance": G_h, "hour": h}
    ml_h = ensemble_average(predict_all_models(...))
    h_ml.append(ml_h)
    h_ideal.append(physics_formula(...))
    h_adj.append(ml_h × combined_factor)
```

Plots four lines: physics formula (gray dotted), ML ensemble (amber), adjusted output (green, filled), AC output (purple dashed). A vertical dashed line marks "Now" (current hour slider value).

Why is irradiance scaled by `sin`? The sun follows an arc across the sky. At 6am and 6pm, light hits at a shallow angle (low irradiance). At noon (hour=12), it hits directly overhead (maximum irradiance). The sine function perfectly models this arc shape.

Tab 2: 🌤️ Irradiance Sensitivity

Sweeps irradiance from 0 to 1100 W/m² in steps of 20, keeping all other inputs fixed. Shows how ML predictions and physics formula diverge as irradiance changes. Useful for understanding how much a cloud passing over impacts output.

Tab 3: 🌡️ Temperature Sensitivity

Sweeps ambient temperature from 0–50°C. Shows the negative slope — as temperature rises, power falls. Useful for estimating summer vs winter output differences.

Tab 4: 📊 Formula vs ML

Loads the test portion of `solar_dataset.csv` and plots physics formula predictions vs actual DC power. The scatter of dots above the diagonal proves the formula overestimates. Shows an R^2 table comparing formula R^2 vs ML R^2 — ML always wins because it was trained on real-world data with losses.

Tab 5: 🔬 Model Performance

Samples 3,000 rows from the training dataset, runs all loaded models on them, and displays:

- A sortable table with R^2, RMSE, MAE, MAPE(>50W) for each model (best cells highlighted green)
- An R^2 bar chart with a red dashed 0.95 target line
- Random Forest feature importance (horizontal bar chart)

---

5. Parameter Cascade — How Changing One Input Affects Everything

Irradiance (G) slider change

```
G changes
  → T_mod changes (NOCT formula: T_mod = T_amb + (noct-20)/800 × G)
    → physics formula P_ideal changes (G and T_mod both in formula)
    → all ML model inputs change (G and T_mod are both features)
      → all_preds change
        → ens_pdc changes
          → adjusted_pdc changes
            → bk (backup) changes
              → all metric cards update
              → gauge updates
              → hourly profile reruns 24× (Tab 1)
              → irradiance sensitivity chart updates (Tab 2)
```

Soiling / Aging / Wiring / Shading sliders change

```
Any loss slider changes
  → total_loss_pct = soiling + aging + wiring + shading
  → combined_factor = (100 - total_loss_pct) / 100
    → adjusted_pdc = ens_pdc × combined_factor   (ML output unchanged, only
adjustment changes)
      → backup hours change
        → "Adjusted Output" metric card updates
        → gauge needle changes
        → loss breakdown expander updates (shows new watt losses per factor)
        → Today's profile adjusted line (h_adj) changes
        → Irradiance sensitivity adjusted line changes
        → Temperature sensitivity ML+losses line changes
  Note: The raw ML predictions (ens_pdc) do NOT change.
        Only the final displayed output changes.
```

Ambient Temperature change

```
T_amb changes
  → T_mod changes (NOCT formula)
    → temp_loss_pct changes = beta × max(0, T_mod-25) × 100
      → thermal loss warning in Module Temperature box updates
        → physics formula P_ideal changes (T_mod is in formula)
        → all ML inputs change (ambient_temp and module_temp are both features)
          → all_preds change → ens_pdc → adjusted_pdc → backup
```

Number of Panels change

```
num_panels changes
  → panel_area_total = panel_area × num_panels  (only in panel_spec dict for ML)
    → all ML predictions scale up proportionally (panel_area is a feature)
      → ens_pdc scales up → adjusted_pdc scales up → backup scales up
  → rated_total = rated_wp × num_panels
    → pct_of_rated changes → status label may change (green/yellow/red)
    → gauge axis range changes (max = rated_total × 1.05)
  Note: physics formula also uses panel_area × num_panels, so p_ideal scales too
```

Panel Preset change

```
Preset selected
  → rated, eff_pct, panel_area, beta_pct, noct_val all update to preset values
    → All downstream effects of changing each of those parameters kick in
simultaneously
      (panel efficiency → physics formula + ML inputs)
      (beta_pct → temperature derating, thermal loss warning, Tab 3 caption)
      (noct → T_mod → everything that depends on T_mod)
      (panel_area → ML panel_area feature + physics formula)
```

Inverter Efficiency change

```
inv_eff changes
  → P_ac = adjusted_pdc × inv_eff  (in compute_backup)
    → E_batt = battery_Wh × batt_dod × inv_eff  (usable battery energy)
      → E_total changes → backup_hours changes
        → AC output line in hourly profile (Tab 1) changes [v × inv_eff for v in
h_adj]
  Note: does NOT change DC power predictions at all
```

Battery sliders (Capacity, DoD) change

```
battery_Wh or batt_dod changes
  → E_batt = battery_Wh × batt_dod × inv_eff
    → E_total = E_panel + E_batt
      → backup = E_total / load_W
        → Backup Runtime metric card updates
        → Bar chart in Chunk 11 updates
```

---

6. The Three Core Fixes Explained

The banner comment at the top of both files references three "FIX" labels. Here's what each means:

FIX 1 — Real-World Loss Factors in Training Data

Problem: Without this, the target variable (`dc_power`) was computed directly from the physics formula with tiny sensor noise. ML models learned to reproduce the formula — which is trivial and pointless (just use the formula instead).

Fix: Four stochastic loss factors (soiling, aging, shading, wiring) are multiplied onto the ideal output. Now `dc_power` cannot be recovered from the formula alone. ML must learn the combined real-world correction.

Effect in app: The loss sliders (sidebar Section 3) let users customize site-specific corrections on top of what ML already accounts for in its statistical average.

FIX 2 — Project Framing (Formula as Baseline, ML as Correction)

Problem: Early framing said "ML replaces the physics formula." This is wrong — if ML just replaced a formula you already have, it's not justified.

Fix: The framing is now: "Physics formula = baseline for ideal conditions. ML = correction for real-world deviations from that ideal." This is academically and practically correct.

Effect in app: Tab 4 ("Formula vs ML") exists specifically to demonstrate this: it shows that the physics formula consistently overestimates (dots above diagonal), while ML predictions sit tightly on the diagonal (accurate).

FIX 3 — MAPE Threshold Raised from 1W to 50W

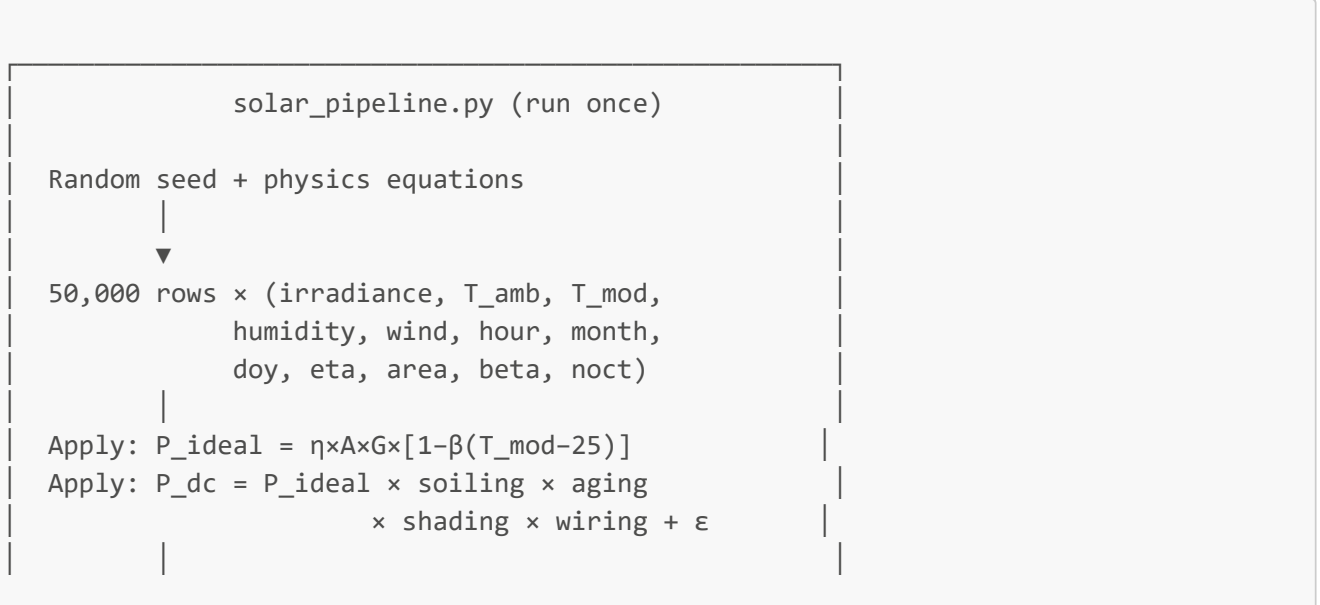
Problem: Dawn/dusk predictions produce huge MAPE values (e.g. 100% error on a 3W prediction) that inflate the reported MAPE to meaningless levels (130–155%). These hours have zero practical significance.

Fix: MAPE is now computed only on rows where actual output > 50W (productive daylight hours). This reduces reported MAPE to 8–15%, which accurately reflects how good the model is during times that matter for energy planning.

Effect in app: Tab 5 labels all MAPE columns with the threshold: `MAPE(>50W)`.

---

7. Data Flow Diagram (End-to-End)



```
│         ▼                                                  │
│  data/solar_dataset.csv  (50,000 rows)                     │
│         │                                                  │
│  MinMaxScaler (fit on 80% train)                           │
│  → models/scaler.pkl                                       │
│         │                                                  │
│         ▼                                                  │
│  Train 5–7 models on scaled features                       │
│  → models/rf_model.pkl                                     │
│  → models/gb_model.pkl                                     │
│  → models/mlp_model.pkl                                    │
│  → models/xgb_model.pkl (or svr_model.pkl)                 │
│  → models/lgb_model.pkl (or et_model.pkl)                  │
│         │                                                  │
│  Evaluate on test 20% → metrics                            │
│  Save charts to outputs/*.png                              │
└──────────────────────────┬─────────────────────────────────┘
                           │  (files on disk)
                           ▼
┌──────────────────────────────────────────────────────────┐
│              app.py (streamlit run app.py)               │
│                                                          │
│  Load scaler.pkl + all model .pkl files (cached)         │
│  Load solar_dataset.csv (cached)                         │
│         │                                                │
│  User adjusts sidebar sliders                            │
│         │ (Streamlit reruns from top on every change)    │
│         ▼                                                │
│  Build [irradiance, T_amb, T_mod, humidity,              │
│         wind, hour, month, doy, eta, area_total,         │
│         beta, noct] → scale with scaler                  │
│         │                                                │
│         ▼                                                │
│  All models predict → array of DC watts                  │
│  Ensemble average = mean of all predictions              │
│  Adjusted = ensemble × (1 − total_loss%)                 │
│         │                                                │
│         ▼                                                │
│  Display: metric cards, gauge, temperature,              │
│           backup chart, 5 analysis tabs                  │
└──────────────────────────────────────────────────────────┘
```

---

8. File & Folder Structure

```
HopefullyFInal/
│
├── solar_pipeline.py       ← Run once: trains and saves everything
├── app.py                  ← Run every time: the Streamlit dashboard
│
├── data/
```

```
│   └── solar_dataset.csv   ← 50,000-row synthetic dataset (generated)
│
├── models/
│   ├── scaler.pkl          ← MinMaxScaler (fit on training data)
│   ├── rf_model.pkl        ← Random Forest model
│   ├── gb_model.pkl        ← Gradient Boosting model
│   ├── mlp_model.pkl       ← MLP / ANN model
│   ├── xgb_model.pkl       ← XGBoost model (if installed)
│   ├── svr_model.pkl       ← SVR fallback (if XGBoost not installed)
│   ├── lgb_model.pkl       ← LightGBM model (if installed)
│   ├── et_model.pkl        ← Extra Trees fallback (if LightGBM not installed)
│   └── best_model.pkl      ← Copy of highest-R² model
│
├── outputs/
│   ├── eda_A_distributions.png      ← EDA Figure A
│   ├── eda_B_correlations.png       ← EDA Figure B
│   ├── eda_C_statistics.png         ← EDA Figure C (stats table)
│   ├── pipeline_diagram.png         ← Architecture flowchart
│   ├── model_comparison.png         ← 8-panel evaluation dashboard
│   ├── results_table.png            ← Ranked metrics table
│   ├── residual_analysis.png        ← Per-model residual histograms
│   └── formula_vs_ml.png            ← FIX 2 validation chart
│
└── docs/
    └── projectsum.md               ← This file
```

Generated for SmartSolar Capstone Project — covers solar_pipeline.py (1122 lines) and app.py (715 lines) in full detail.